Istacherni: From Retrieval to Fine-Tuning — A Complete Technical Report

Chapter 1: Project Overview and Motivation

The **Istacherni** project addresses a fundamental challenge in the Algerian legal domain: citizens, lawyers, and scholars need fast, accurate access to the correct legal article when facing a specific legal question. Traditional keyword search fails because Arabic legal text is morphologically complex, and a question like *"What is the penalty for theft?"* may not share a single word with the actual article that defines it.

The solution is a **Retrieval-Augmented Generation (RAG)** system — a pipeline that first retrieves the most relevant legal articles and then uses a Large Language Model (LLM) to synthesize a well-grounded Arabic answer from those articles. Over the course of this project, we built and iterated through four distinct retrieval architectures (BM25, Dense, Hybrid, Graph, and Agentic RAG) and then pushed accuracy further through domain-specific fine-tuning of the BGE-M3 embedding model.

The primary evaluation metric is **Mean Reciprocal Rank (MRR)** on a golden dataset of 127 Arabic legal questions, each annotated with the correct ground-truth article(s) from the Algerian law corpus.

Chapter 2: Data Foundation

Before any retrieval can happen, we need a structured, queryable corpus of Algerian law. This chapter covers how we went from raw PDF issues of the Algerian Official Gazette to a clean, normalized JSON dataset.

2.1 Data Ingestion from JORADP — `scraper.py`

The Algerian Official Gazette (*الجريدة الرسمية* — JORADP) is the authoritative source for all Algerian legislation. It publishes decrees, laws, and amendments in PDF format at `joradp.dz`. The `scraper.py` script automates the full pipeline from download to structured JSON in three stages.

Stage 1 — Download (`JORADPScraper` class):
The scraper builds a URL from a year and issue number template (`A{year}{issue:03d}.pdf`) and downloads the PDF using `requests` with retry logic (via `urllib3.Retry`) for transient HTTP errors (429, 500–504). Because JORADP uses a certificate chain that Windows cannot verify natively, SSL verification is disabled. Downloaded PDFs are cached locally under `dataset/raw/joradp_pdfs/{year}/`, so re-runs do not re-download existing issues.

Stage 2 — Text Extraction (`PDFExtractor` class):
Text is extracted page-by-page using `pdfplumber`. This works well for digitally-generated PDFs. However, older Gazette issues (pre-2000) are scanned images. When `pdfplumber` extracts fewer than 50 characters from a page, the system falls back to OCR: it converts the page to a 300 DPI image via `pdf2image` and runs `pytesseract` with the Arabic language model (`lang="ara"`). The first and last few lines of each issue are captured as `header` and `footer` metadata, which help Gemini infer the law name and section structure.

Stage 3 — Gemini Parsing (`GeminiParser` class):
Each page's raw text is sent to `gemini-1.5-flash` with a detailed Arabic system prompt that instructs the model to extract every legal article and return a structured JSON array. Each article object must conform to a precise schema including: `id`, `law_domain`, `law_name`, `article_number`, `text.original`, `text_explanation`, `summary`, `legal_conditions_summary`, `penalties_summary`, `keywords`, `legal_type` (substantive / procedural / penal), `norm_type` (mandatory / supplementary / discretionary), and `relations.related_articles`. A 4.5-second delay between requests prevents hitting the 15 RPM quota limit.

2.2 Data Loading and Normalization — `bm25_loader.py`

This module is the universal entry point for all retrieval systems. It recursively scans the `dataset/raw/` directory for all `.json` files and loads them into a unified flat list of canonical article dictionaries.

Why normalization is necessary:
The dataset was built incrementally from multiple sources (the JORADP scraper, manually curated files, and data contributed across different sessions). This means the same field can appear under different keys: the article text may be stored as `text.original`, `text_original`, or inside a nested dict like `{"original": {"1": "...", "4": "..."}}` (multi-article format). The `_extract_text()` function handles all three variants transparently, joining sub-article texts with spaces when they appear as a numbered dict.

The `_normalize_article()` function:
Every raw dict is passed through this function, which maps fields to a canonical schema:

Canonical Field	Source(s) in raw JSON
`id`	`id` or synthesized from `article_number`
`law_name`	`law_name` or `law_type`
`law_domain`	`law_domain` or `law_type`
`article_number`	Converted to string; lists are joined with `-`
`text_original`	Via `_extract_text()`
`text_explanation`	`text_explanation`, `definition`, or `summary`
`keywords`	`keywords` or `tags`
`penalties_summary`	`penalties_summary`
`legal_conditions_summary`	`legal_conditions_summary`

Articles with no extractable text are discarded. A sophisticated deduplication mechanism tracks every `(id, text_original, text_explanation)` triplet seen. If the same `id` appears with genuinely different content (a revised version of the article), it is kept with a versioned suffix (e.g., `DZ_PENAL_ART_350_1`), ensuring both versions are searchable.

Chapter 3: BM25 Sparse Retrieval

BM25 (Best Match 25) is a probabilistic ranking function that scores documents based on term frequency and inverse document frequency. It excels at exact keyword matching — critical for queries that mention a specific legal term, law name, or penalty type.

3.1 Arabic Tokenisation — `bm25_indexer.py`

Arabic text presents unique challenges for tokenisation compared to Latin-script languages:

- **Diacritics (tashkeel):** The same word can be written with or without vowel marks (e.g., كَتَبَ vs كتب). A naïve tokeniser would treat these as different tokens.
- **Letter variants:** The letters آ, أ, إ, and ا are all variants of alef. ة (taa marbuta) is often interchangeable with ه. ى (alef maqsura) is often confused with ي.
- **No spaces between prefixes:** Arabic attaches conjunctions and prepositions directly to words (e.g., وَالْقَانُون — "and the law").

The `tokenize_arabic()` function addresses all of these:

1. Strips all diacritics by removing Unicode range U+064B–U+065F and the tatweel character U+0640.
2. Normalises alef variants: `[ا → [اآإأ`; taa marbuta: `ة → ه`; alef maqsura: `ى → ي`.
3. Splits on any character that is not an Arabic letter, Latin letter, or digit.
4. Discards tokens shorter than 2 characters (removes single-character prefixes like `و`, `ل`, `ب`).

3.2 Document Building and Indexing

Before tokenisation, each article is converted to a rich searchable string by `build_document_text()`. This function concatenates: a structured header `[law_name - المادة art_num]`, the title, the original text, the summary, the legal conditions summary, the penalties summary, and the keywords field — **repeated twice**. This keyword repetition is a deliberate technique: BM25 scores based on term frequency, so doubling the keywords boosts the retrieval weight of the most legally relevant terms without requiring any model retraining.

The tokenised corpus is passed to `BM25Okapi` from the `rank_bm25` library. The resulting BM25 object and the article corpus (maintaining positional correspondence) are both serialised to disk as `bm25_index.pkl` and `bm25_corpus.pkl`. On subsequent startups, the index loads from cache in under a second rather than rebuilding from scratch.

3.3 Query Execution — `bm25_retriever.py`

At query time, the user's question is tokenised with the same `tokenize_arabic()` function, ensuring vocabulary alignment between documents and queries. `BM25Okapi.get_scores()` computes a relevance score for every document in the corpus in a single vectorised operation. The top-K articles are then returned by sorting indices with `numpy.argsort`. Articles with a score of zero or below are filtered out, as they share no tokens with the query.

The BM25 retriever is deliberately standalone — it imports no embedding models, requires no GPU, and makes no network calls, making it extremely fast (typically under 100ms for the full corpus).

3.4 Answer Generation — `bm25_generator.py`

Once articles are retrieved, they are formatted into a structured context block by `_build_context()`, which includes the law name, article number, title, full text, conditions summary, and penalties summary for each retrieved article. This context is injected into a prompt sent to `gemini-2.5-flash` with a strict Arabic system instruction: answer using only the provided articles, cite every article by name and number, and admit when information is insufficient. Rate-limit errors (HTTP 429) are handled with exponential backoff and a retry loop.

Chapter 4: Dense Semantic Retrieval

While BM25 excels at keyword matching, it fails on semantic queries — when the user describes a situation without using the exact legal terminology. Dense retrieval maps both queries and documents into a high-dimensional vector space where semantic similarity is measured by cosine distance.

4.1 Embedding Model — `bge_embedder.py`

The system uses **BAAI/bge-m3**, a state-of-the-art multilingual embedding model that supports Arabic, French, and English — all three languages present in the Algerian legal corpus. The model is loaded locally using `sentence-transformers` (`SentenceTransformer`).

Document representation — `build_searchable_string()`:
Each article is converted to a comprehensive string that joins: the structured header `[law_name - المادة art_num]`, the title, all keywords, the original text, the explanation, and the summary — separated by `|` delimiters. Including all metadata fields maximises the semantic surface area that the embedding captures.

Query representation:
For queries, a BGE-M3-specific prefix is prepended: `"Represent this query for retrieving relevant documents: {query}"`. This asymmetric retrieval pattern is recommended by the BGE-M3 authors because the model was trained with separate query and document encoders. The prefix signals to the model to produce a query-optimised embedding rather than a document embedding.

Batch embedding:
Articles are embedded in configurable batches (default: 16 for CPU, 64 for GPU) using `model.encode()` with `normalize_embeddings=True`. L2-normalised embeddings allow cosine similarity to be computed efficiently as a dot product.

4.2 Vector Index — `bge_indexer.py`

The embeddings are stored in a local **ChromaDB** persistent vector database. ChromaDB uses the **HNSW (Hierarchical Navigable Small World)** algorithm for approximate nearest-neighbour search, configured with cosine similarity (`hnsw:space: cosine`). The indexer embeds all articles in batches and adds them to the collection with their full metadata (id, law_name, article_number, title, text, etc.) for retrieval. If a batch fails (e.g., due to an encoding error on a malformed article), the system falls back to article-by-article insertion, skipping only the problematic entry.

4.3 Dense Retrieval — `bge_retriever.py`

At query time, the query is embedded using the same model and the resulting vector is compared against all stored embeddings via ChromaDB's `collection.query()`. This returns the top-K nearest neighbours by cosine similarity along with their stored metadata, which is then reconstructed into the same article dict format used by BM25, enabling seamless fusion.

Chapter 5: Hybrid RAG — Combining the Best of Both Worlds

Neither BM25 nor dense retrieval alone achieves the accuracy required for legal work. BM25 misses semantically similar articles that use different vocabulary; dense retrieval sometimes fails on rare or highly specific legal terms. The Hybrid RAG system combines both, then applies a neural reranker to produce the final ranked list.

5.1 Reciprocal Rank Fusion — `hybrid_retriever.py`

The core fusion algorithm is **Reciprocal Rank Fusion (RRF)**, defined as:

```
score(d) = Σ_{r ∈ rankers}  weight(r) × 1 / (k + rank(d, r))
```

where `k = 60` is a constant that dampens the influence of very high ranks. The key insight of RRF is that it is rank-based, not score-based: it doesn't matter that BM25 produces scores in the range [0, 20] while cosine similarity produces scores in [0, 1]. Both are converted to a common rank-based currency.

Implementation details:
The system fetches the top 100 candidates from each retriever independently. For each article, it accumulates the RRF score from both rankers using the article's unique `id` (or a composite `law_name__article_number` key as fallback). Articles appearing in both retriever results receive contributions from both, naturally surfacing high-quality

matches. The merged dict is then sorted in descending RRF score order, yielding up to 200 unique candidates that are passed to the reranker.

5.2 Cross-Encoder Reranker — `reranker.py`

The reranker is the final accuracy gatekeeper. Unlike the bi-encoder approach used for dense retrieval (where query and document are embedded independently), a cross-encoder evaluates the `[Query, Document]` pair **jointly**, allowing it to model fine-grained interactions between the two.

Unified BGE-M3 as Reranker:
After experimentation, we replaced a standard cross-encoder with the fine-tuned `bge-m3-unified` model. This model supports three retrieval heads simultaneously: Dense, Sparse (lexical), and ColBERT (multi-vector). The reranker calls `model.compute_score(pairs, weights_for_different_modes=[0.4, 0.2, 0.4])`, fusing the ColBERT and Dense scores at 40% weight each and the Sparse score at 20%.

Document construction for reranking:
Each candidate article is serialized into a rich text string: `[law_name - المادة art_num] title. original_text explanation summary keywords`. This rich representation ensures the cross-encoder has full context when assessing relevance.

Why this ordering matters: The reranker is computationally expensive (it processes each pair individually through a transformer), so it is only applied to the top 100 RRF candidates, not the full corpus. This two-stage architecture (cheap retrieval → expensive reranking) is the standard pattern in production IR systems.

Chapter 6: Graph RAG — Relational Knowledge Retrieval

Legal articles do not exist in isolation. Article 350 of the Penal Code may reference Article 14; a commercial law article may be the basis for a civil liability claim. Graph RAG exploits these explicit structural relationships to retrieve articles that are legally relevant even if they share no keywords with the query.

6.1 Knowledge Graph Construction — `graph_builder.py`

The graph is built using **NetworkX** as a directed graph (`DiGraph`). It contains four types of nodes and five types of edges, all derived purely from the structured JSON metadata — no external API calls are required.

Node types:

Node Type	ID Format	Derived From
`article`	The article's JSON `id`	One per law article
`concept`	`CONCEPT:{keyword}`	Each keyword in the `keywords` field
`domain`	`DOMAIN:{law_domain}`	The `law_domain` field
`penalty`	`PENALTY:{class}`	Regex classification of `penalties_summary`

Edge types:

Edge	Direction	Meaning
`HAS_KEYWORD`	article → concept	This article covers this legal concept
`IN_DOMAIN`	article → domain	This article belongs to this law domain
`HAS_PENALTY`	article → penalty	This article imposes this penalty class
`RELATED_TO`	article → article	Explicit cross-reference in `relations.related_articles`
`SAME_LAW`	article ↔ article	Both articles belong to the same law

Penalty classification:
The `_classify_penalty()` function uses a set of regex patterns to map the freeform `penalties_summary` text to canonical penalty classes (e.g., إعدام → الإعدام, حبس → الحبس, غرامة → الغرامة المالية). This transforms unstructured text into structured graph nodes that can be directly queried.

PageRank pre-computation:
After the graph is built, PageRank is computed using `nx.pagerank(G, alpha=0.85)` and cached to disk. PageRank assigns a global importance score to every article based on how many other articles reference it via `RELATED_TO` and `SAME_LAW` edges. Foundational articles of a law (e.g., definitions and general principles) naturally receive high PageRank scores. This score is used as a tiebreaker during retrieval.

6.2 Multi-Hop Graph Traversal — `graph_retriever.py`

Step 1 — Query Term Extraction:
The query is tokenised with the same Arabic normaliser used in BM25. Stop-words (ما, هل, من, في, etc.) are filtered out, leaving only meaningful legal terms.

Step 2 — Seed Node Discovery (Hop 0):
The algorithm scans all `concept`, `penalty`, and `domain` nodes in the graph and checks whether any query term is a substring of the node's label, or vice versa. This substring matching (rather than exact matching) handles morphological variations — a query term سرقة (theft) will match a concept node سرقة بالإكراه (aggravated theft).

Step 3 — Hop 1 (Direct Article Retrieval):
For each seed node, the algorithm walks backward along the graph's edges (finding all `predecessors`) to collect article nodes that declared that keyword, penalty, or domain. It also does a forward walk to catch any article→article edges. Each article's hit count is tracked: an article matched by 3 different seed nodes gets a score of 3.

Additionally, the algorithm scans all article nodes directly and checks if any query terms appear in their `keywords` or `title` fields, adding to their hit count. This catches articles that are relevant but whose concept nodes didn't appear as seeds.

Step 4 — Hop 2 (Structural Expansion):
For each Hop-1 article, the algorithm follows `RELATED_TO` edges to find articles that are explicitly cross-referenced. These get a reduced weight of 1.0 (vs 2.0 for Hop-1 articles). This expansion ensures that if Article 350 is relevant, Article 14 (which it references) is also surfaced as context.

Step 5 — Final Scoring:
The combined score for each article is: `hits × weight + PageRank × 10`. The PageRank multiplier of 10 ensures that authoritative, frequently-referenced articles are consistently surfaced even with fewer keyword hits.

Chapter 7: Agentic RAG — Iterative Reasoning with Tool Use

For complex, ambiguous queries — where the legal domain is unclear or where the answer requires synthesizing information across multiple articles — a single-pass retrieval is insufficient. The Agentic RAG system uses an LLM as a reasoning engine that can iteratively search the knowledge base until it has gathered enough information to answer confidently.

7.1 Tool Declarations

The agent is powered by **Gemini 2.5 Flash** with function-calling. Three tools are exposed to the model:

Tool	Description	When the agent uses it
`search_articles`	Full-corpus BM25 keyword search	First search when domain is unknown
`filter_by_domain`	BM25 search filtered to a specific legal domain	When domain is identified (Penal / Civil / Labor / Commercial)
`get_article_by_id`	Fetch full text of one specific article by ID	When the agent needs to read a specific article in full

7.2 The Agentic Loop — `agentic_agent.py`

The agent's reasoning loop operates as follows:

1. The user's question is wrapped with a detailed Arabic system instruction that defines the agent's role as a digital Algerian legal expert.
2. The full conversation history (system prompt + user question) is sent to Gemini.
3. Gemini responds with either: (a) a list of function calls (tool invocations) or (b) a final text answer.
4. If tool calls are returned, each is dispatched to `_execute_tool()`, which calls the actual KB function and returns a JSON result string.
5. The tool results are added to the conversation history as a `tool` role message and the loop repeats (up to `MAX_TOOL_ROUNDS = 5` iterations).
6. If Gemini produces no tool calls, it has synthesized its final answer and the loop exits.

This iterative architecture allows the agent to **self-correct**: if the first `search_articles` call returns Civil Law articles when the question is about Penal Law, the agent can recognize this, call `filter_by_domain` with `domain="Penal Law"` on the next round, and get the correct articles.

Rate-limit handling:
The `_call_with_retry()` function wraps every Gemini API call with exponential-backoff retry logic. If a 429 error is received, it parses the `retryDelay` from the error message and waits exactly that long before retrying.

Chapter 8: Fine-Tuning BGE-M3 for Algerian Legal Retrieval

The off-the-shelf BGE-M3 model has strong Arabic capabilities but was not trained on Algerian legal text. Our evaluation revealed a systematic failure mode: the model consistently confuses articles from different legal codes that use similar vocabulary. For example, a query about commercial fraud might rank an article from the Civil Code higher than the correct Penal Code article. The fine-tuning pipeline was designed specifically to fix these cross-code confusions.

8.1 Deep Hard Negative Mining — `mine_hard_negatives.py`

The key to effective embedding fine-tuning is the quality of the training triplets `(query, positive_article, hard_negative_article)`. A **hard negative** is an article that the current model ranks highly for a given query but which is actually the *wrong* answer. It is "hard" because the model is confidently wrong about it.

Version 2 innovations:

Retrieval Depth: The retrieval pool was expanded to 200 candidates (from 20 in v1) to ensure we find true hard negatives even for queries where the model gets the top-5 mostly right.

Rank-Shift Sampling Window:
Rather than sampling negatives arbitrarily, the `_neg_rank_window()` function adapts the sampling window based on where the golden article ranks in the retrieval results:

- **Golden at rank 1:** Sample negatives from ranks 10–50. These are "distractor zone" articles that look relevant but aren't.
- **Golden at ranks 2–9:** Sample from the ranks immediately adjacent to the golden (±1) up to rank 50, targeting the exact confusion zone.
- **Golden at rank ≥10:** The model is severely confused — sample from all top-50 non-golden articles to maximize signal.

Strict Δ-Filter:
Only negatives within a score delta of $\Delta \leq 0.10$ from the golden article's rerank score are accepted. This ensures the negative is genuinely "hard" — close enough in score to train the model on the exact boundary. Queries with no passing negatives are flagged for manual review and excluded from training.

Cross-Code Bonus:
The `law_code()` function maps each article to a canonical legal code (penal, civil, commercial, labour, etc.). Hard negatives from a *different* legal code than the golden article are sorted first, directly targeting the cross-code confusion problem.

Output format:
Each triplet is written as a JSONL line:

```
{"query": "...", "pos": ["full article text"], "neg": ["hard negative text"]}
```

8.2 Fine-Tuning the Bi-Encoder — `train_embedder.py`

This script fine-tunes the BGE-M3 base model using the `sentence-transformers` training API.

Loss function — `MultipleNegativesRankingLoss`:
This loss takes a batch of (query, positive) pairs and treats all other positives in the batch as in-batch negatives. Combined with our explicit hard negatives, it simultaneously:

- Pulls the query embedding closer to the positive article embedding.
- Pushes the query embedding away from the hard negative embedding.
- Uses all other queries' positives as additional negatives for free (in-batch negatives).

Training configuration:

- Learning rate: `2e-5` — conservative enough to preserve BGE-M3's inherent multilingual knowledge.
- Batch size: 1 (with gradient accumulation) to fit on a 6GB VRAM GPU.
- Epochs: 3 with a 10% warmup phase.
- Query prefix: `"Represent this query for retrieving relevant documents: {query}"` is prepended to all queries, as recommended by BAAI.

8.3 Fine-Tuning the Cross-Encoder Reranker — `train_crossencoder.py`

While the bi-encoder determines which articles enter the top-K candidate pool, the cross-encoder makes the final ranking decision. Fine-tuning it on legal triplets makes it an aggressive domain-specific judge.

Model: `BAAI/bge-reranker-v2-m3` loaded as a sequence classification model (`AutoModelForSequenceClassification`) with a single output logit (relevance score).

Loss function — `MarginRankingLoss(margin=0.3)`:
Given a positive pair `(query, positive)` and a negative pair `(query, negative)`, the loss is:

```
L = max(0, -(pos_score - neg_score) + 0.3)
```

This forces the model to score the positive at least 0.3 higher than the negative. The margin of 0.3 creates a decisive separation, not just a small preference.

Training loop:
Both `(query, positive)` and `(query, negative)` are tokenised independently and forward-passed through the same model to get `pos_scores` and `neg_scores`. The loss is computed between them and backpropagated with `AdamW` at `lr=2e-5` with a linear warmup scheduler.

8.4 Unified Multi-Head Fine-Tuning — `train_bge_m3_unified.ps1`

BGE-M3's most powerful capability is its three retrieval heads: **Dense** (vector similarity), **Sparse** (lexical matching, like a neural BM25), and **ColBERT** (token-level late interaction). Standard fine-tuning with `sentence-transformers` only optimises the Dense head. The Unified fine-tuning pipeline from BAAI optimises all three heads simultaneously.

Launch script (`train_bge_m3_unified.ps1`):
The training uses `torch.distributed.run` with a single GPU (simulating the DDP environment that the BAAI trainer requires). Key parameters:

- `--unified_finetuning True` — activates all three heads.
- `--use_self_distill True` — the Dense head's output is used as a teacher signal to guide the Sparse and ColBERT heads, improving consistency across the three retrieval modes.
- `--temperature 0.02` — a very low temperature creates sharper, more confident contrastive learning signals.
- `--normlized True` — enables L2 normalization for cosine-compatible scores.
- `--negatives_cross_device` — shares negatives across distributed processes for maximum in-batch diversity.

The `run_unified_training.py` wrapper:
Because BAAI's trainer assumes a distributed multi-GPU environment, this Python wrapper manually sets the `MASTER_ADDR`, `MASTER_PORT`, `WORLD_SIZE`, `RANK`, and `LOCAL_RANK` environment variables to simulate a single-node single-GPU DDP setup. It then directly invokes `FlagEmbedding.finetune.embedder.encoder_only.m3.__main__.main()`, bypassing the need for a separate process launcher on Windows.

After training, the resulting `bge-m3-unified` model was integrated into both `bge_embedder.py` (for ChromaDB indexing) and `reranker.py` (for cross-encoder scoring), giving the full pipeline the benefit of all three retrieval heads at both the candidate generation and reranking stages.

Chapter 9: Evaluation Framework

9.1 The Golden Dataset

All evaluation is performed against a curated golden dataset (`algerian_law_ragas_dataset_v3.json`) of 127 Arabic legal questions. Each question is annotated with one or more ground-truth articles in the format `"قانون العقوبات - المادة 350"` (law name + article number). This dataset covers all major legal domains: Penal, Civil, Commercial, Labor, and Administrative law.

9.2 Evaluation Metrics — `evaluate_retrieval.py`

The `match_articles()` function compares the retrieved chunks against the ground-truth articles. Matching is done by checking: (1) the `article_number` strings are equal, and (2) the `law_name` strings overlap (substring match), handling minor variations in how the same law is named across files.

Metrics computed:

Metric	Formula	What it measures
Recall@30	`matched / total_expected`	What fraction of required articles were found in top 30
Hit Rate	`hits / total_questions`	Was at least one correct article retrieved?
Precision@1	`correct_at_rank_1 / total`	Was the top result the correct article?
MRR	`Σ (1 / rank_of_first_hit) / N`	How high up was the first correct article?

MRR is the primary metric because it directly measures whether the system put the *right article first* — the key requirement for a system that will present a single answer to the user. A high Recall@30 with low MRR means the system finds the right article but buries it — unacceptable for production use.

9.3 Results Summary

The fine-tuning pipeline produced measurable, consistent improvements across all metrics, with MRR showing the largest gains. The cross-code confusion (ranking a Civil article #1 for a Penal query) was dramatically reduced by the combination of cross-code hard negatives and the cross-encoder margin loss. The final deployed system uses the `bge-m3-unified` model for both embedding and reranking, delivering the highest MRR on the 127-question benchmark.

Conclusion

The Istachemi project demonstrates a systematic approach to building a high-accuracy domain-specific RAG system for Algerian law. Starting from raw PDF scraping through the JORADP pipeline, through four complementary retrieval architectures, to a fully custom fine-tuning pipeline built around hard negative mining, each component was designed to address a specific limitation of the previous approach. The result is a system that retrieves the legally correct article at Rank 1 with high reliability — a prerequisite for any trustworthy legal assistant.